

Linked List: Overview and Objectives

Abstract data types and linked lists

1. What this unit is about

An abstract data type, or ADT, is a way of describing a collection of data together with the operations allowed on it, without saying how those operations are actually built. We call this a wall: the program on one side asks for an operation such as add or remove, and the data structure on the other side carries it out, without the program needing to know what is happening behind the wall. That separation is called information hiding, and it is one of the main reasons ADTs are useful.

The linked list is the first ADT we study in depth, and one of the most fundamental data structures in the course. A linked list is a collection of items, called nodes, where each node holds some data and a link to the next node. Understanding how a linked list is put together, and how its variations differ, sets up almost everything that follows this unit, including stacks, queues, trees, and graphs, all of which reuse the same idea of items linked to their neighbors.

As with recursion, we work in **two languages at once**. Computer Science students write **C/C++**; CIS students write **Java**. The linked list idea is identical in both: a node holding data and a reference to the next node. Where the languages genuinely differ, most visibly in how Java hides a pointer's address while C/C++ exposes it directly, we call it out.

Class time favors drawings and live code over slides. Read ahead, try tracing a few nodes on paper yourself, and come ready to think out loud.

2. Learning objectives

By the end of the unit you should be able to:

Remember and understand

- Define an ADT and state the features it provides, including modularity and information hiding.
- Discuss the components of an ADT: the collection of data and the set of operations defined on it.
- List the common operations shared by most ADTs, such as create, add, retrieve, remove, destroy, determine emptiness, and determine size.
- Discuss the terminology of a linked list, including node, link, head, tail, predecessor, and successor.
- Summarize the purpose of linked lists and discuss the concept of a linear data structure.
- Discuss the implementation types of a linked list, array-based and pointer-based, and the types of linked list, singly linked, doubly linked, circular, and circular doubly linked.

Analyze

- Differentiate among the components of an ADT and explain how they relate to one another.
- Understand the difference between an ADT and a class or object, and explain why the distinction matters.
- Identify examples of ADTs beyond the linked list, such as stacks, queues, tables, trees, and graphs.
- Compare array-based and pointer-based implementations, and compare the four linked list types against one another.

Apply

- Diagram a linked list solution by hand, including the node, link, head, and tail.
- Code common linked list operations, and new ones as problems call for them.
- Respond to a stated problem by carrying out the correct linked list operation or algorithm.
- Generate a linked list code solution to a new problem.

Evaluate

- Select the most efficient strategy for a given linked list operation, and justify the choice against the alternatives.